



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO®



Instituto **Tecnológico**®
de Aguascalientes

Instituto Tecnológico de Aguascalientes

Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

Wifi 3 ESP32

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 26 de mayo de 2026

Índice

Introducción	3
Código ArduinoIDE	4
Conclusiones	16
Enlace Github	17



Introducción

En esta práctica se implementó un sistema de comunicación entre un ESP32 S3 y una base de datos SQL Server utilizando una arquitectura basada en servicios web. El microcontrolador se conecta a una red WiFi y envía información mediante peticiones HTTP hacia una API desarrollada en NodeJS, la cual actúa como intermediaria entre el dispositivo y la base de datos alojada en un servidor Docker con MSSQL. El objetivo de esta implementación es comprender el flujo de datos desde un dispositivo IoT hasta un sistema de almacenamiento centralizado, sentando las bases para la integración de sensores y la recopilación de información en el proyecto final.



Código ArduinoIDE

// Código para ESP32 S3 - Conexión a MSSQL vía servidor REST

// Librerías necesarias:

// - WiFi (incluida en Arduino IDE para ESP32)

// - HTTPClient (incluida en Arduino IDE para ESP32)

Librerías necesarias para el funcionamiento de la practica

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
#include <ArduinoJson.h>
```

Configuración necesaria para entrar en al red de la base de datos y la ip del dispositivo donde se encuentra el servidor

```
// ===== CONFIGURACIÓN =====
```

```
const char* ssid = "Totalplay-9CA5";
```

```
const char* password = "9CA5C414gV4Qs844";
```

```
const char* apiUrl = "http://192.168.100.11:3000/api";
```

```
const char* testUrl = "http://192.168.100.11:3000/api/test";
```

```
const char* insertUrl = "http://192.168.100.11:3000/api/insert";
```

```
// Pines de trabajo del ESP32 S3 |Ajusta estos valores según tu proyecto
```

```
// Pines de trabajo del ESP32 S3 |Ajusta estos valores según tu proyecto
```

```
// ===== FUNCIONES LED INTEGRADO=====
```

Al no contar con un LED RGB integrado se opto por hacer un sistema de blinks

```
#define LED_PIN 2
```

```
void setupLED() {
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW); // Apagado inicial
}
```

Cuando la conexión esta en curso hace un blink rapido

```
void blinkFast() { // Conexión en curso
    digitalWrite(LED_PIN, HIGH);
    delay(200);
    digitalWrite(LED_PIN, LOW);
    delay(200);
}
```

Cuando la conexión es exitosa hace blink mas lento

```
void blinkSlow() { // Conexión exitosa
    digitalWrite(LED_PIN, HIGH);
    delay(1000);
    digitalWrite(LED_PIN, LOW);
    delay(1000);
}
```

Si hay algun error en la conexión el blink es doble y repetido

```
void blinkError() { // Error de conexión
    for (int i = 0; i < 2; i++) {
        digitalWrite(LED_PIN, HIGH);
        delay(150);
        digitalWrite(LED_PIN, LOW);
        delay(150);
    }
}
```



```

}

delay(500); // Pausa entre ciclos
}

// ===== FUNCIONES WiFi =====

```

Funcion que conecta el ESP32 a la red wifi

```

void connectToWiFi() {

    Serial.println("\n\nInicializando conexión WiFi...");

    Serial.print("Conectando a: ");

    Serial.println(ssid);

```

Configura el Esp32 como un cliente que se conectara al router con los datos del ssid y contraseña definidos arriba

```

    WiFi.mode(WIFI_STA);

    WiFi.begin(ssid, password);

```

Funcion que cuenta los intentos para conectarse a la red wifi.

Se tiene un delay de 500 ms entre cada intento a conectarse. Este bucle terminara cuando se conecte a la red wifi o hasta que hayan pasado 20 intentos de conectarse.

```

int attemp = 0;

while (WiFi.status() != WL_CONNECTED && attemp < 20) {

    delay(500);

    Serial.print(".");

    attemp++;

    blinkFast();

}

```

Si la conexión es exitosa pasara a dar un aviso y cambiara el blink a lento.
de lo contrario avisara del error y parpadeara doble el led

```
if (WiFi.status() == WL_CONNECTED) {  
    Serial.println("\n✓ Conectado a WiFi");  
    Serial.print("IP: ");  
    Serial.println(WiFi.localIP());  
    blinkSlow();  
} else {  
    Serial.println("\nX Error: No se pudo conectar a WiFi");  
    blinkError();  
}  
}
```

// ===== FUNCIONES HTTP =====

```
bool testDatabaseConnection() {  
    mensaje serial avisando de la conexión a la base de datos  
    Serial.println("\nProbando conexión a MSSQL...");
```

Se hace un objeto http y se prueba la url almacenada en testUrl. Se especifica con el Content-Type que se trabajaran con datos de tipo json.

```
HTTPClient http;  
http.begin(testUrl);  
http.addHeader("Content-Type", "application/json");
```

Se solicita una petición Get al servidor y se guardará en la variable `httpCode` que almacenará los códigos de la solicitud

```
int httpCode = http.GET();
```

Analiza la respuesta de la solicitud GET y da mensajes de la respuesta del servidor

```
if (httpCode == 200) {  
    String response = http.getString();  
    Serial.println("✓ Conexión a BD exitosa");  
    Serial.println(response);  
    http.end();  
    return true;  
} else {  
    Serial.print("X Error conectando a BD: ");  
    Serial.println(httpCode);  
    http.end();  
    return false;  
}  
}
```

Esta función es la encargada de hacer los inserts a la base de datos con los valores de nombre, valor y temperatura.

```
bool insertDataToDatabase(const char* nombre, int valor, float temperatura) {  
    Serial.println("\nEnviando INSERT a MSSQL...");
```


Se crea un cliente http con la url almacenada en insertUrl e igualmente se especifica que se usaran datos json

```
HTTPClient http;  
  
http.begin(insertUrl);  
  
http.addHeader("Content-Type", "application/json");
```

Se crea un documento json con los campos que se insertaran en la base de datos

```
// Crear JSON con los datos  
  
DynamicJsonDocument doc(200);  
  
doc["nombre"] = nombre;  
  
doc["valor"] = valor;  
  
doc["temperatura"] = temperatura;
```

Esta function convierte los datos json en un string para poder ser enviados por http

```
String jsonString;  
  
serializeJson(doc, jsonString);  
  
  
Serial.print("Datos enviados: ");  
  
Serial.println(jsonString);
```

Envia los datos al servidor con una petición POST

```
int httpCode = http.POST(jsonString);
```

Codigos de respuesta que avisan del insert correcto o un error

```
if (httpCode == 200) {
```

```

String response = http.getString();

Serial.println("✓ INSERT exitoso");

Serial.println(response);

http.end();

return true;
} else {

Serial.print("X Error en INSERT: ");

Serial.println(httpCode);

String response = http.getString();

Serial.println(response);

http.end();

return false;

}

}

// ===== SETUP =====

void setup() {

Serial.begin(115200);

delay(1000);

Serial.println("\n\n=== ESP32 S3 MSSQL DEBUG ===");

```



Inicia la function del led integrado para avisar del estado.

```
setupLED();
```

```
// Conectar a WiFi
```

Usa la funcion para conectar a wifi. Si no se puede conectar a wifi avisa del error y vuelve a intentar la conexion

```
connectToWiFi();
```

```
if (WiFi.status() != WL_CONNECTED) {
```

```
    blinkError();
```

```
    Serial.println("No se pudo conectar a WiFi. Reiniciando en 5 segundos...");
```

```
    delay(5000);
```

```
    ESP.restart();
```

```
}
```

Una vez hecha la conexión, procede a conectarse a la base de datos mediante un bucle con limite de 10 intentos cada 500ms.

```
Serial.println("\nProbando conexión a base de datos...");
```

```
bool connectionOk = false;
```

```
for (int i = 0; i < 10; i++) {
```

```
    blinkFast();
```

Si la conexion es correcta se rompe el bucle

```
    if (testDatabaseConnection()) {
```

```
        connectionOk = true;
```

```

    break;
}
delay(500);
}

```

Si la conexión no es correcta avisa del error y reinicia el esp32

```

if (!connectionOk) {
    blinkError();

    Serial.println("No se pudo conectar a la BD. Verifique la configuración.");
    delay(5000);
    ESP.restart();
}

```

Se intenta hacer el insert a la base de datos

```

// Hacer un INSERT de prueba

Serial.println("\nRealizando INSERT de prueba...");

```

Se manda a llamar la función insertDataToDatabase con unos valores predeterminados. El estado de la inserción es dado a conocer en la terminal serial

```

if (insertDataToDatabase("ESP32_S3_Test", 42, 23.5)) {
    blinkSlow();

    Serial.println("\n✓✓✓ TODO CORRECTO - LED VERDE ✓✓✓");
} else {
    blinkError();

    Serial.println("\nX X X ERROR EN INSERT - LED ROJO X X X");
}
}

```

```
// ===== LOOP =====
```

```
void loop() {
```

```
    // Mantener el LED en el estado actual
```

```
    // Cada 30 segundos, hacer otro INSERT para demostración
```

Una vez hecho el insert predeterminado, se realizara un insert cada 30 segundos

```
    static unsigned long lastInsert = 0;
```

```
    if (millis() - lastInsert > 30000) {
```

```
        lastInsert = millis();
```

```
        Serial.println("\n\n=== Enviando INSERT periódico ===");
```

Se crea un valor aleatorio para las variables que se enviaran: valor y temperatura

```
        int valorRandom = random(0, 100);
```

```
        float temperaturaRandom = 20.0 + (random(0, 50) / 10.0);
```

Se Vuelve a llamar a la funcion para insertar los datos a la base de datos con el nombre predeterminado, y las variables que acaban de generarse de manera aleatoria. Posteriormente da el estatus de la inserción mediante la terminal Serial

```
        if (insertDataToDatabase("ESP32_Periodic", valorRandom, temperaturaRandom)) {
```

```
            blinkFast();
```

```
            Serial.println("✓ INSERT periódico exitoso");
```

```

} else {

    blinkError();

    Serial.println("X INSERT periódico fallido");

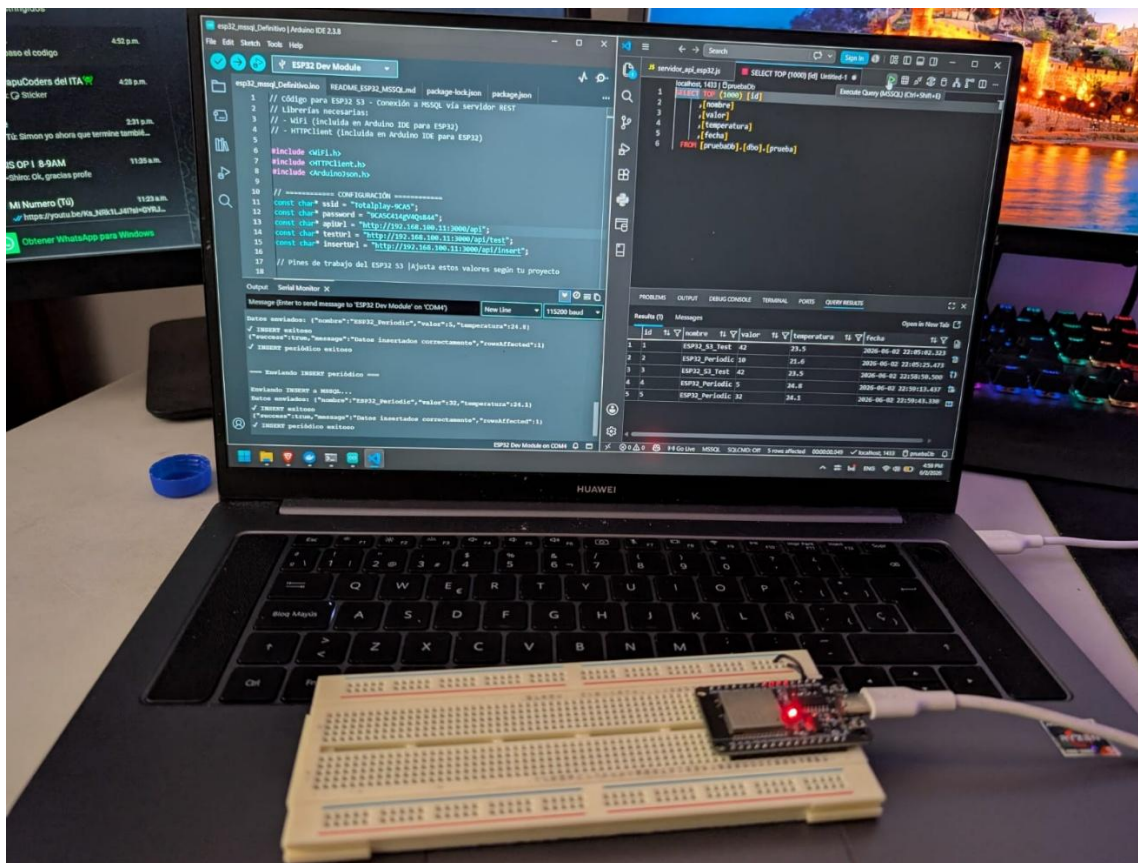
}

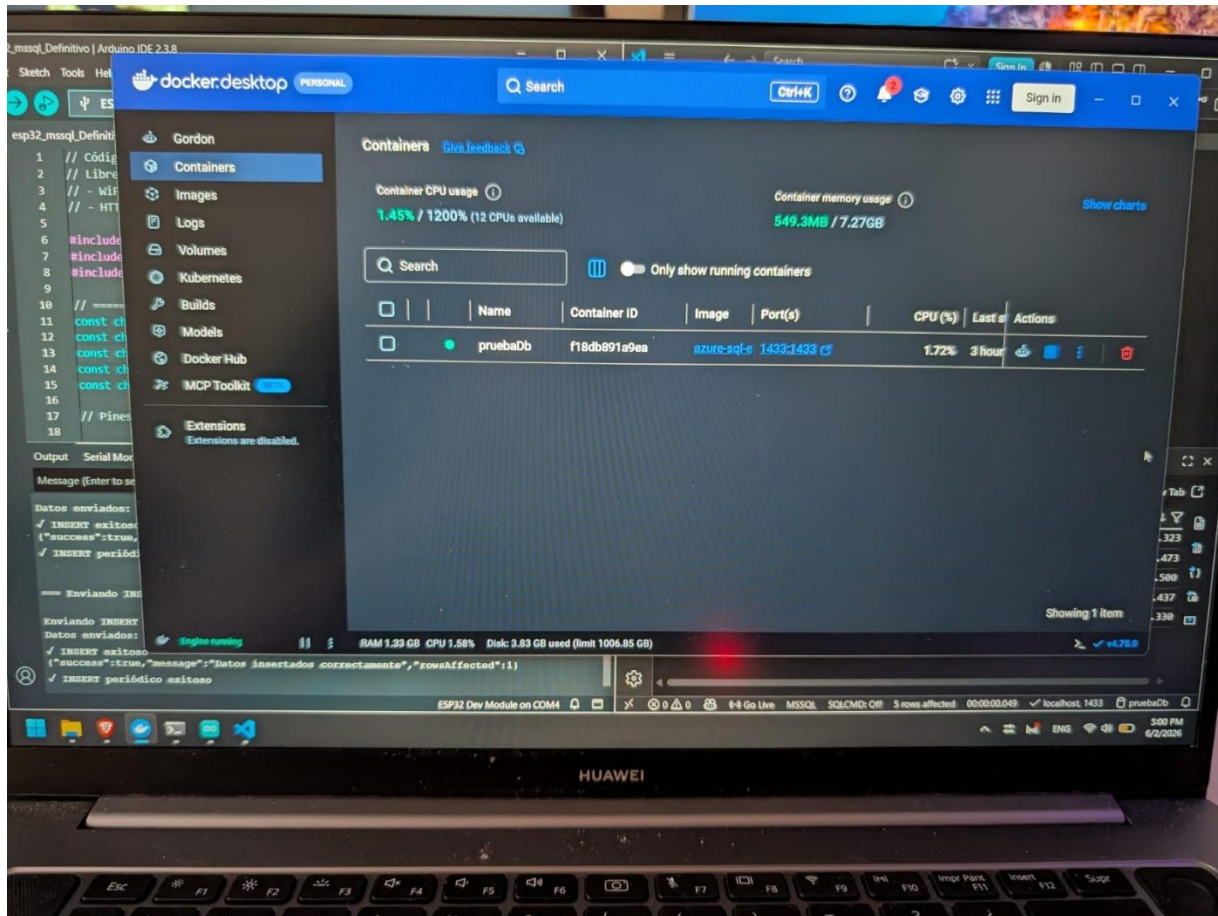
}

delay(100);

}

```





Conclusiones

Martín de Jesús Ramírez Rodríguez:

Se logró establecer una comunicación funcional entre el ESP32 S3 y una base de datos SQL Server mediante una API REST desarrollada en NodeJS. A través de la conexión WiFi y el uso de solicitudes HTTP, el microcontrolador pudo verificar la disponibilidad del servidor y realizar inserciones de datos en la base de datos de manera exitosa. Esta práctica permitió comprender la integración de tecnologías IoT con sistemas de gestión de bases de datos, así como la importancia de las APIs como intermediarias para el intercambio seguro y estructurado de información. Los resultados obtenidos proporcionan una base sólida para la incorporación de sensores reales y el desarrollo del proyecto final.

Pedroza Pérez Emmanuel:

En la práctica aprendimos a transmitir datos de algún dispositivo controlado por el esp32 por medio de una red wifi que fue procesada por el servidor de la base de datos.

Esto nos da las bases para realizar todo tipo de aplicaciones como el controlar y registrar la humedad de un invernadero o detectar cambios en el ambiente y clima.

Cristian Alexis Santacruz Rodarte:

Con la práctica se pudo realizar la conexión hacia un servidor a través de las redes Wifi creadas, con esta conexión se enviaron datos hacia una base de datos SQL, también se hizo uso de APIs externas lo cual ayudó a la integración de datos en el servidor, la práctica nos ayudó para entender conexiones por medios de Wifi hacia servidores externos.



Enlace Github

A continuación, se adjunta el enlace con el código ArduinoIDE expuesto, así como el video de evidencia y documentación:

<https://github.com/MartinITAgs/WIFI-ESP32>

